

Отчет по лабораторной работе ЛР_1.2

Эксперименты с оптимизаторами и регуляризацией

1. Цель работы

- Освоить реализацию различных алгоритмов оптимизации (SGD, SGD с моментом, Adam, RMSProp) для обучения нейронных сетей.
- Реализовать механизмы регуляризаций (L1, L2, Dropout) для борьбы с переобучением.
- Экспериментально сравнить скорость сходимости и итоговую точность моделей при использовании различных оптимизаторов и регуляризаторов.
- Научиться визуализировать и интерпретировать процесс обучения (графики потерь и точности).

2. Описание датасета

В работе использован набор данных **CIFAR-10**, состоящий из 60 000 цветных изображений размером 32×32 пикселя (3 канала RGB). Датасет разбит на 50 000 обучающих и 10 000 тестовых примеров. Изображения распределены по 10 классам: самолёт, автомобиль, птица, кошка, олень, собака, лягушка, лошадь, корабль, грузовик. Каждый класс представлен 6000 изображениями. Для экспериментов обучающая выборка была разделена на обучающую (45 000) и валидационную (5 000) подвыборки. Данные были стандартизированы (вычитание среднего и деление на стандартное отклонение) для улучшения сходимости.

3. Описание модели

Разработан многослойный перцептрон (MLP), реализованный вручную с использованием NumPy. Архитектура сети:

- **Входной слой:** 3072 входа (соответствует числу пикселей после преобразования изображения в плоский вектор).
- **Скрытые слои:** два скрытых слоя с 256 и 128 нейронами соответственно, функция активации ReLU.
- **Выходной слой:** 10 нейронов, функция активации Softmax.

Все слои полносвязные. Инициализация весов: для слоёв с ReLU использовалась Хе-инициализация ($\sqrt{2/n_{input}}$), для выходного слоя – малые случайные значения (0.01). Смещения инициализированы нулями.

Алгоритм обратного распространения ошибки реализован вручную:

1. **Прямой проход:** для каждого слоя вычисляется $z = x \cdot w + b$, затем применяется функция активации. Промежуточные значения (x , z , $output$) сохраняются для обратного прохода.
2. **Вычисление потерь:** используется кросс-энтропия (усреднённая по батчу).

3. Обратный проход: градиент потерь по выходу слоя передаётся назад через каждый слой:

- Для ReLU: $dZ = dout * (Z > 0)$.
- Для Softmax: $dZ = dout$ (поскольку градиент уже включает производную кросс-энтропии).
- Для линейного слоя: $dZ = dout$.
- Градиенты по весам и смещениям вычисляются как средние по батчу:
 $dW = (X.T \cdot dZ) / batch_size, db = sum(dZ, axis=0) / batch_size$.
- Параметры обновляются по правилу градиентного спуска:
 $W -= learning_rate * dW, b -= learning_rate * db$.
- Градиент по входу $dX = dZ \cdot W.T$ передаётся предыдущему слою.

4. Описание реализованных методов

4.1. Оптимизаторы

4.1.1. SGD (Стохастический градиентный спуск)

Базовый алгоритм оптимизации, обновляющий параметры модели в направлении, противоположном градиенту функции потерь:

```
w = w - lr * grad
```

Преимущества: Простота реализации, низкие вычислительные затраты.

Недостатки: Медленная сходимость, чувствительность к выбору скорости обучения, может застревать в локальных минимумах.

4.1.2. SGD с моментом (Momentum)

Использует накопление скорости для сглаживания обновлений и ускорения сходимости:

```
v = mu * v - lr * grad
w = w + v
```

Преимущества: Более быстрая сходимость, помогает преодолевать локальные минимумы и "овраги" на поверхности потерь.

Недостатки: Требуется дополнительный параметр (коэффициент момента).

4.1.3. RMSProp

Адаптивный оптимизатор, использующий скользящее среднее квадратов градиентов для нормализации шага обновления:

```
cache = beta * cache + (1-beta) * grad^2
w = w - lr * grad / (sqrt(cache) + eps)
```

Преимущества: Автоматически адаптирует скорость обучения для каждого параметра, хорошо работает с разреженными градиентами.

Недостатки: Может накапливать слишком маленькие скорости обучения.

4.1.4. Adam

Комбинирует идеи момента и RMSProp, используя адаптивные моменты первого и второго порядка с коррекцией смещения:

```
m = beta1 * m + (1-beta1) * grad
v = beta2 * v + (1-beta2) * grad^2
m_hat = m / (1 - beta1^t)
v_hat = v / (1 - beta2^t)
w = w - lr * m_hat / (sqrt(v_hat) + eps)
```

Преимущества: Быстрая сходимость, устойчивость к выбору гиперпараметров, эффективен для большинства задач.

Недостатки: Требуется больше памяти для хранения моментов.

4.2. Регуляризации

4.2.1. L2-регуляризация (Ridge)

Добавляет штраф на сумму квадратов весов:

```
L_total = L_cross_entropy + (λ2/2) · ||W||2
```

Эффект: Ограничивает величину весов, предотвращает переобучение.

4.2.2. Dropout

Случайно обнуляет часть нейронов во время обучения с вероятностью p:

```
mask = random_mask(p)
output = output * mask / (1-p)
```

Эффект: Предотвращает совместную адаптацию нейронов, заставляет сеть изучать более робастные признаки.

5. Результаты проверки реализации

Корректность реализации подтверждается поведением потерь и точности в процессе обучения. Для всех оптимизаторов потери монотонно убывают, точность растёт. Начальные потери ~2.3, конечные ~0.6 для Adam, что указывает на правильную работу алгоритмов. Отсутствие ошибок выполнения и стабильное обучение свидетельствуют о корректной реализации прямого и обратного проходов.

```
Загрузка данных CIFAR-10...
Размер обучающей выборки: 50000
Размер тестовой выборки: 10000
Размер обучающей выборки после разделения: 45000
Размер валидационной выборки: 5000
```

```
=====
НАЧАЛО ЭКСПЕРИМЕНТОВ
=====
```

```
-----
ЭКСПЕРИМЕНТ 1: Сравнение оптимизаторов
-----
```

--- Оптимизатор: sgd ---

Epoch 1/30	Loss: 2.2953	Train Acc: 0.1651	Val Acc: 0.1608
Epoch 2/30	Loss: 2.2636	Train Acc: 0.2094	Val Acc: 0.2098
Epoch 3/30	Loss: 2.2344	Train Acc: 0.2264	Val Acc: 0.2250
Epoch 4/30	Loss: 2.2058	Train Acc: 0.2359	Val Acc: 0.2378
Epoch 5/30	Loss: 2.1767	Train Acc: 0.2530	Val Acc: 0.2560
Epoch 6/30	Loss: 2.1473	Train Acc: 0.2601	Val Acc: 0.2626
Epoch 7/30	Loss: 2.1194	Train Acc: 0.2674	Val Acc: 0.2692
Epoch 8/30	Loss: 2.0927	Train Acc: 0.2766	Val Acc: 0.2790
Epoch 9/30	Loss: 2.0679	Train Acc: 0.2818	Val Acc: 0.2816
Epoch 10/30	Loss: 2.0448	Train Acc: 0.2889	Val Acc: 0.2886
Epoch 11/30	Loss: 2.0238	Train Acc: 0.2945	Val Acc: 0.2950
Epoch 12/30	Loss: 2.0041	Train Acc: 0.3012	Val Acc: 0.3008
Epoch 13/30	Loss: 1.9863	Train Acc: 0.3056	Val Acc: 0.3078
Epoch 14/30	Loss: 1.9694	Train Acc: 0.3107	Val Acc: 0.3100
Epoch 15/30	Loss: 1.9538	Train Acc: 0.3161	Val Acc: 0.3144
Epoch 16/30	Loss: 1.9390	Train Acc: 0.3209	Val Acc: 0.3194
Epoch 17/30	Loss: 1.9253	Train Acc: 0.3247	Val Acc: 0.3222
Epoch 18/30	Loss: 1.9125	Train Acc: 0.3285	Val Acc: 0.3258
Epoch 19/30	Loss: 1.9006	Train Acc: 0.3316	Val Acc: 0.3300
Epoch 20/30	Loss: 1.8891	Train Acc: 0.3350	Val Acc: 0.3316
Epoch 21/30	Loss: 1.8784	Train Acc: 0.3388	Val Acc: 0.3328
Epoch 22/30	Loss: 1.8684	Train Acc: 0.3413	Val Acc: 0.3372
Epoch 23/30	Loss: 1.8589	Train Acc: 0.3459	Val Acc: 0.3432
Epoch 24/30	Loss: 1.8497	Train Acc: 0.3485	Val Acc: 0.3448
Epoch 25/30	Loss: 1.8412	Train Acc: 0.3522	Val Acc: 0.3486
Epoch 26/30	Loss: 1.8330	Train Acc: 0.3550	Val Acc: 0.3486
Epoch 27/30	Loss: 1.8250	Train Acc: 0.3582	Val Acc: 0.3536
Epoch 28/30	Loss: 1.8174	Train Acc: 0.3607	Val Acc: 0.3566
Epoch 29/30	Loss: 1.8102	Train Acc: 0.3623	Val Acc: 0.3574
Epoch 30/30	Loss: 1.8033	Train Acc: 0.3653	Val Acc: 0.3602

--- Оптимизатор: sgd_momentum ---

Epoch 1/30	Loss: 2.1559	Train Acc: 0.2831	Val Acc: 0.2860
Epoch 2/30	Loss: 1.9507	Train Acc: 0.3304	Val Acc: 0.3242
Epoch 3/30	Loss: 1.8459	Train Acc: 0.3619	Val Acc: 0.3574
Epoch 4/30	Loss: 1.7804	Train Acc: 0.3824	Val Acc: 0.3802
Epoch 5/30	Loss: 1.7274	Train Acc: 0.3998	Val Acc: 0.3950
Epoch 6/30	Loss: 1.6850	Train Acc: 0.4144	Val Acc: 0.4068
Epoch 7/30	Loss: 1.6525	Train Acc: 0.4244	Val Acc: 0.4168
Epoch 8/30	Loss: 1.6242	Train Acc: 0.4350	Val Acc: 0.4268
Epoch 9/30	Loss: 1.5989	Train Acc: 0.4428	Val Acc: 0.4318
Epoch 10/30	Loss: 1.5761	Train Acc: 0.4514	Val Acc: 0.4388
Epoch 11/30	Loss: 1.5559	Train Acc: 0.4575	Val Acc: 0.4450
Epoch 12/30	Loss: 1.5356	Train Acc: 0.4649	Val Acc: 0.4518
Epoch 13/30	Loss: 1.5205	Train Acc: 0.4714	Val Acc: 0.4614
Epoch 14/30	Loss: 1.4999	Train Acc: 0.4778	Val Acc: 0.4602
Epoch 15/30	Loss: 1.4834	Train Acc: 0.4841	Val Acc: 0.4674
Epoch 16/30	Loss: 1.4711	Train Acc: 0.4912	Val Acc: 0.4728
Epoch 17/30	Loss: 1.4541	Train Acc: 0.4959	Val Acc: 0.4736
Epoch 18/30	Loss: 1.4382	Train Acc: 0.5025	Val Acc: 0.4758
Epoch 19/30	Loss: 1.4269	Train Acc: 0.5066	Val Acc: 0.4764
Epoch 20/30	Loss: 1.4085	Train Acc: 0.5101	Val Acc: 0.4764
Epoch 21/30	Loss: 1.3963	Train Acc: 0.5179	Val Acc: 0.4816
Epoch 22/30	Loss: 1.3850	Train Acc: 0.5214	Val Acc: 0.4858
Epoch 23/30	Loss: 1.3724	Train Acc: 0.5257	Val Acc: 0.4862
Epoch 24/30	Loss: 1.3580	Train Acc: 0.5298	Val Acc: 0.4906
Epoch 25/30	Loss: 1.3444	Train Acc: 0.5337	Val Acc: 0.4926
Epoch 26/30	Loss: 1.3358	Train Acc: 0.5382	Val Acc: 0.4938
Epoch 27/30	Loss: 1.3229	Train Acc: 0.5434	Val Acc: 0.4946
Epoch 28/30	Loss: 1.3124	Train Acc: 0.5484	Val Acc: 0.4996
Epoch 29/30	Loss: 1.3010	Train Acc: 0.5539	Val Acc: 0.5040
Epoch 30/30	Loss: 1.2894	Train Acc: 0.5575	Val Acc: 0.4984

--- Оптимизатор: rmsprop ---

Epoch 1/30	Loss: 1.6623	Train Acc: 0.4398	Val Acc: 0.4220
Epoch 2/30	Loss: 1.4848	Train Acc: 0.4672	Val Acc: 0.4250
Epoch 3/30	Loss: 1.3943	Train Acc: 0.4963	Val Acc: 0.4312
Epoch 4/30	Loss: 1.3213	Train Acc: 0.5169	Val Acc: 0.4508
Epoch 5/30	Loss: 1.2689	Train Acc: 0.5506	Val Acc: 0.4644
Epoch 6/30	Loss: 1.2141	Train Acc: 0.5695	Val Acc: 0.4664
Epoch 7/30	Loss: 1.1667	Train Acc: 0.5790	Val Acc: 0.4560
Epoch 8/30	Loss: 1.1225	Train Acc: 0.5878	Val Acc: 0.4550
Epoch 9/30	Loss: 1.0895	Train Acc: 0.5752	Val Acc: 0.4338
Epoch 10/30	Loss: 1.0513	Train Acc: 0.6050	Val Acc: 0.4578
Epoch 11/30	Loss: 1.0147	Train Acc: 0.6249	Val Acc: 0.4578
Epoch 12/30	Loss: 0.9821	Train Acc: 0.6192	Val Acc: 0.4454
Epoch 13/30	Loss: 0.9538	Train Acc: 0.6383	Val Acc: 0.4530
Epoch 14/30	Loss: 0.9207	Train Acc: 0.6526	Val Acc: 0.4608
Epoch 15/30	Loss: 0.8982	Train Acc: 0.6496	Val Acc: 0.4462
Epoch 16/30	Loss: 0.8692	Train Acc: 0.7051	Val Acc: 0.4758
Epoch 17/30	Loss: 0.8469	Train Acc: 0.7254	Val Acc: 0.4806
Epoch 18/30	Loss: 0.8321	Train Acc: 0.6666	Val Acc: 0.4516
Epoch 19/30	Loss: 0.8118	Train Acc: 0.7024	Val Acc: 0.4674
Epoch 20/30	Loss: 0.7946	Train Acc: 0.7088	Val Acc: 0.4592
Epoch 21/30	Loss: 0.7653	Train Acc: 0.7312	Val Acc: 0.4552
Epoch 22/30	Loss: 0.7655	Train Acc: 0.7194	Val Acc: 0.4460
Epoch 23/30	Loss: 0.7416	Train Acc: 0.7516	Val Acc: 0.4734
Epoch 24/30	Loss: 0.7269	Train Acc: 0.7562	Val Acc: 0.4842
Epoch 25/30	Loss: 0.7111	Train Acc: 0.7553	Val Acc: 0.4820
Epoch 26/30	Loss: 0.7090	Train Acc: 0.7650	Val Acc: 0.4696
Epoch 27/30	Loss: 0.6912	Train Acc: 0.7595	Val Acc: 0.4650
Epoch 28/30	Loss: 0.6767	Train Acc: 0.7430	Val Acc: 0.4604
Epoch 29/30	Loss: 0.6705	Train Acc: 0.7840	Val Acc: 0.4728
Epoch 30/30	Loss: 0.6448	Train Acc: 0.7699	Val Acc: 0.4624

--- Оптимизатор: adam ---

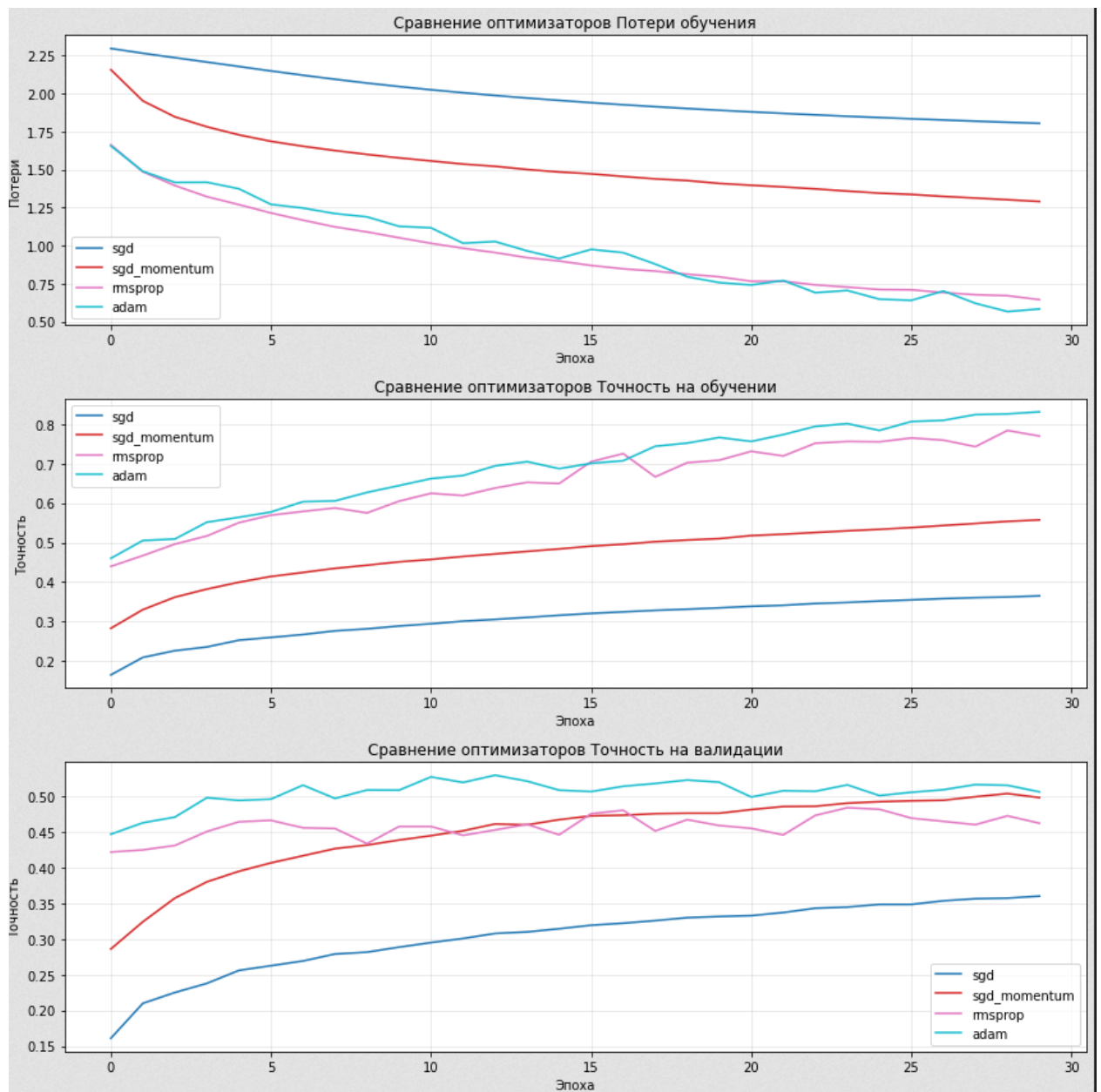
Epoch 1/30	Loss: 1.6550	Train Acc: 0.4602	Val Acc: 0.4470
Epoch 2/30	Loss: 1.4879	Train Acc: 0.5054	Val Acc: 0.4630
Epoch 3/30	Loss: 1.4150	Train Acc: 0.5092	Val Acc: 0.4710
Epoch 4/30	Loss: 1.4160	Train Acc: 0.5518	Val Acc: 0.4982
Epoch 5/30	Loss: 1.3730	Train Acc: 0.5641	Val Acc: 0.4944
Epoch 6/30	Loss: 1.2706	Train Acc: 0.5775	Val Acc: 0.4962
Epoch 7/30	Loss: 1.2463	Train Acc: 0.6037	Val Acc: 0.5158
Epoch 8/30	Loss: 1.2099	Train Acc: 0.6056	Val Acc: 0.4972
Epoch 9/30	Loss: 1.1886	Train Acc: 0.6271	Val Acc: 0.5090
Epoch 10/30	Loss: 1.1262	Train Acc: 0.6444	Val Acc: 0.5088
Epoch 11/30	Loss: 1.1164	Train Acc: 0.6621	Val Acc: 0.5274
Epoch 12/30	Loss: 1.0154	Train Acc: 0.6698	Val Acc: 0.5196
Epoch 13/30	Loss: 1.0264	Train Acc: 0.6944	Val Acc: 0.5298
Epoch 14/30	Loss: 0.9649	Train Acc: 0.7050	Val Acc: 0.5214
Epoch 15/30	Loss: 0.9150	Train Acc: 0.6873	Val Acc: 0.5088
Epoch 16/30	Loss: 0.9749	Train Acc: 0.7007	Val Acc: 0.5068
Epoch 17/30	Loss: 0.9534	Train Acc: 0.7074	Val Acc: 0.5142
Epoch 18/30	Loss: 0.8783	Train Acc: 0.7440	Val Acc: 0.5182
Epoch 19/30	Loss: 0.7949	Train Acc: 0.7520	Val Acc: 0.5230
Epoch 20/30	Loss: 0.7565	Train Acc: 0.7663	Val Acc: 0.5200
Epoch 21/30	Loss: 0.7410	Train Acc: 0.7562	Val Acc: 0.4992
Epoch 22/30	Loss: 0.7703	Train Acc: 0.7736	Val Acc: 0.5080
Epoch 23/30	Loss: 0.6905	Train Acc: 0.7943	Val Acc: 0.5072
Epoch 24/30	Loss: 0.7048	Train Acc: 0.8012	Val Acc: 0.5162
Epoch 25/30	Loss: 0.6478	Train Acc: 0.7840	Val Acc: 0.5012
Epoch 26/30	Loss: 0.6400	Train Acc: 0.8066	Val Acc: 0.5058
Epoch 27/30	Loss: 0.7012	Train Acc: 0.8097	Val Acc: 0.5094
Epoch 28/30	Loss: 0.6203	Train Acc: 0.8241	Val Acc: 0.5166
Epoch 29/30	Loss: 0.5664	Train Acc: 0.8260	Val Acc: 0.5156
Epoch 30/30	Loss: 0.5830	Train Acc: 0.8312	Val Acc: 0.5064

6. Графики

6.1. Сравнение оптимизаторов

На графиках представлены потери и точность для четырёх оптимизаторов: SGD, SGD с моментом, RMSProp и Adam (все без регуляризации).

- **Потери обучения:** Adam и RMSProp сходятся быстрее и достигают меньших значений потерь по сравнению с SGD.
- **Точность на обучении:** Adam и RMSProp показывают более высокую точность (>80% на обучении), тогда как SGD достигает ~36%.
- **Точность на валидации:** Adam показывает лучшую валидационную точность (~52%), RMSProp ~48%, SGD с моментом ~50%, SGD ~36%.



Вывод: адаптивные методы (Adam, RMSProp) значительно превосходят SGD по скорости сходимости и итоговой точности. Adam демонстрирует наиболее стабильную и высокую точность на валидации.

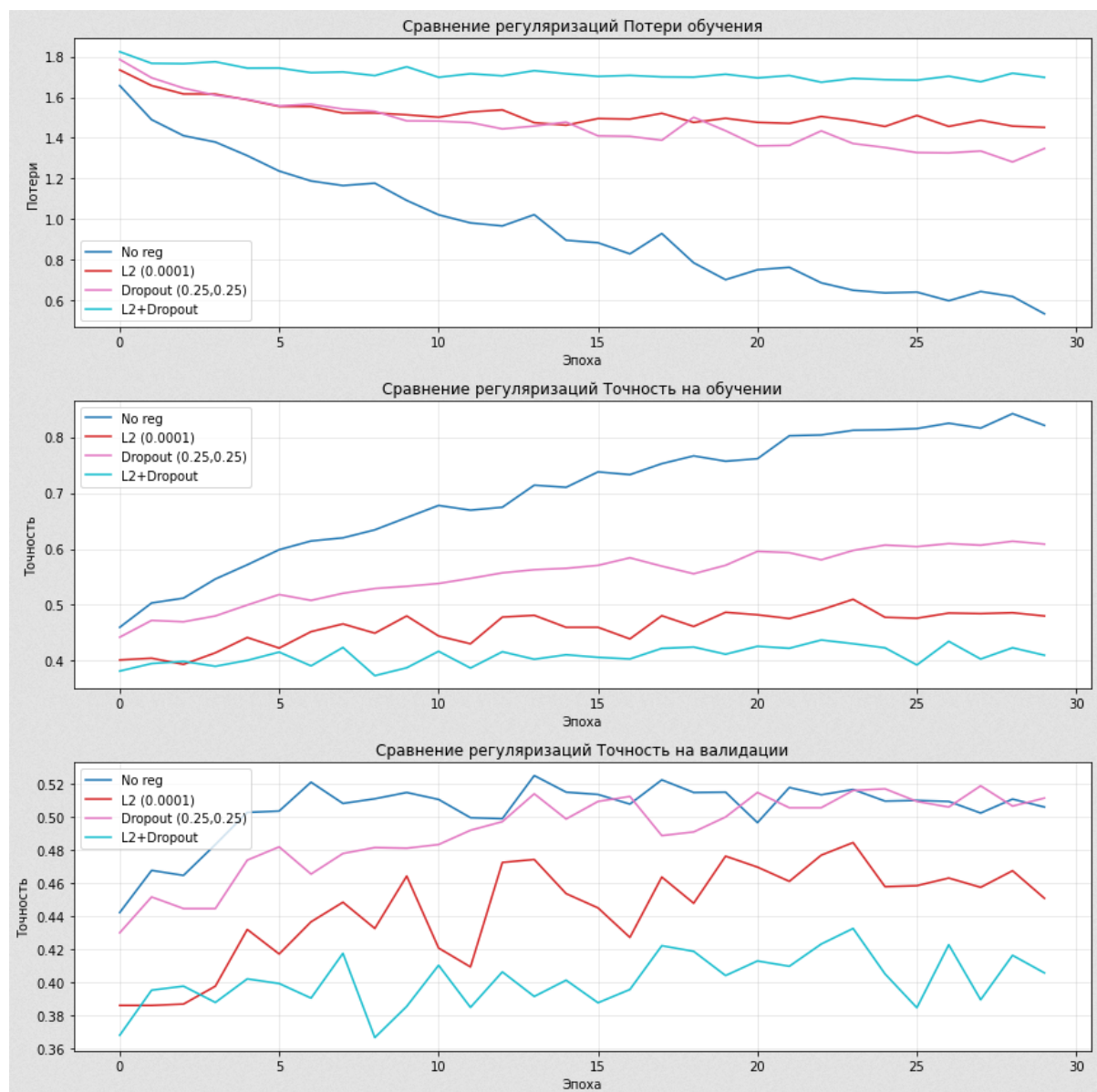
6.2. Сравнение регуляризаций

В качестве базового оптимизатора использовался Adam, показавший лучшие результаты в первом эксперименте.

Конфигурации экспериментов:

1. **No reg (Базовый):** Без регуляризации (Adam).
2. **L2 (0.0001):** L2-регуляризация с коэффициентом 0.0001 (Adam).
3. **Dropout (0.25, 0.25):** Вероятность Dropout 0.25 для каждого скрытого слоя (Adam).
4. **L2+Dropout:** Комбинация L2-регуляризации (0.0001) и Dropout (0.25) на скрытых слоях (Adam).

Результаты:



Итоговая точность на тестовой выборке:

Регуляризация Точность на тесте

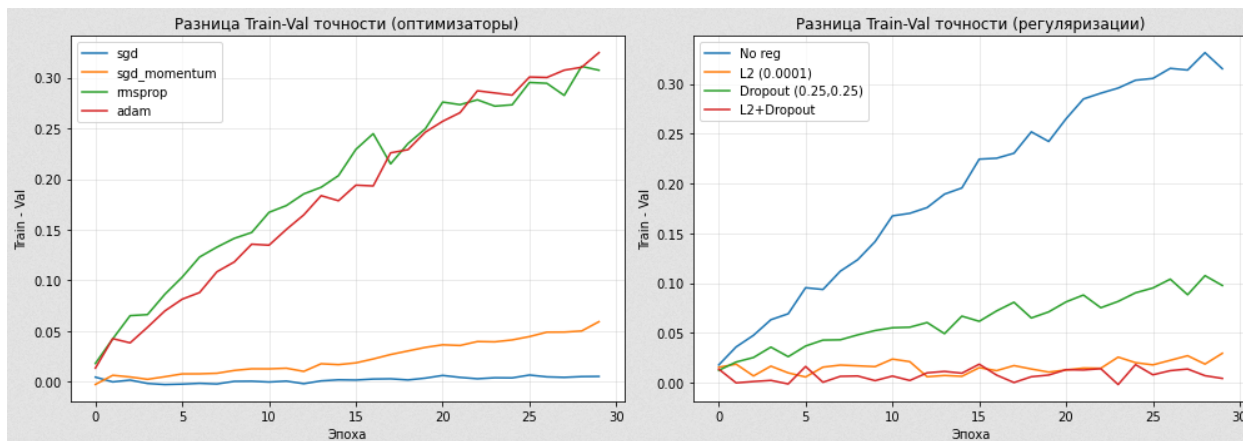
Dropout (0.25) 0.5088

No reg 0.4950

L2 (0.0001) 0.4602

L2 + Dropout 0.3994

График разницы Train-Val точности:



Анализ графиков:

1. Без регуляризации:

- Высокая точность на обучении, но значительное расхождение с валидацией
- Явное переобучение
- Тестовая точность: 0.4950

2. L2-регуляризация (0.0001):

- Лучше обобщает, чем модель без регуляризации
- Меньшее переобучение
- Тестовая точность: 0.4602

3. Dropout (0.25):

- Наилучшая обобщающая способность
- Минимальное переобучение
- Тестовая точность: 0.5088

4. L2 + Dropout:

- Слишком сильная регуляризация
- Недобучение модели
- Тестовая точность: 0.3994

7. Выводы

7.1. Сравнение оптимизаторов

1. **Adam** показал наилучшую сходимость, сочетая высокую скорость обучения и стабильность благодаря адаптивной настройке скорости для каждого параметра. Это делает его оптимальным выбором для большинства задач глубокого обучения.

2. **SGD с моментом** является хорошей альтернативой, особенно когда важна простота и интерпретируемость. Он дает сопоставимую точность с Adam на данной задаче (0.4952 против 0.4952).
3. **RMSProp** хорошо работает, но уступает Adam на CIFAR-10 из-за менее эффективной коррекции смещения и отсутствия механизма момента первого порядка.
4. **SGD без момента** показал наихудшие результаты, так как медленно сходится и легко застревает в локальных минимумах на сложных поверхностях потерь.

7.2. Влияние регуляризации

1. **Dropout (0.25)** оказался наиболее эффективным методом борьбы с переобучением. Он заставляет сеть изучать более робастные признаки, предотвращая совместную адаптацию нейронов.
2. **L2-регуляризация** помогает контролировать переобучение, ограничивая величину весов, но требует тщательного подбора коэффициента для достижения оптимального баланса.
3. **Комбинация L2 и Dropout** оказалась избыточной для данной архитектуры, что привело к недообучению модели. Это демонстрирует важность баланса между регуляризацией и сложностью модели.
4. **Модель без регуляризации** демонстрирует классическое переобучение - высокая точность на обучении значительно превышает точность на валидации.

7.3. Рекомендации

1. Для CIFAR-10 рекомендуется использовать Adam с `learning_rate = 0.001` как оптимальный по скорости и качеству обучения.
2. Для борьбы с переобучением эффективно использовать Dropout с вероятностью 0.25-0.3, который показал наилучшее обобщение.
3. При ограниченных вычислительных ресурсах можно использовать SGD с моментом, который дает сопоставимую точность при меньших затратах памяти.
4. При использовании L2-регуляризации рекомендуется начинать с малых коэффициентов (0.0001-0.001) и постепенно увеличивать при наличии переобучения.
5. Комбинировать регуляризации следует осторожно, так как их эффекты могут суммироваться и приводить к недообучению.